Nisandi Disara - 23001951

Mobile Application Development

Practical – 10

01)

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Student Accommodation Form',
      theme: ThemeData(
        primarySwatch: Colors.amber, // Special requirement: screen color amber
      ), ThemeData
      home: const StudentFormScreen(),
    ); MaterialApp
  }
}

class StudentFormScreen extends StatefulWidget {
  const StudentFormScreen({super.key});

  @override
  State<StudentFormScreen> createState() => _StudentFormScreenState();
}

class _StudentFormScreenState extends State<StudentFormScreen> {
  // Form key - used to control validation
  final _formKey = GlobalKey<FormState>();

```

```dart
  final _formKey = GlobalKey<FormState>();

  // Controllers - used to read the text field values
  final TextEditingController _usernameController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();
  final TextEditingController _accommodationDetailsController =
  TextEditingController();

  // Radio button state - whether the student has a laptop
  String? _hasLaptop = 'Yes'; // default value

  // Dropdown state - whether the student is an IS student
  String? _isStudent = 'Yes'; // default value

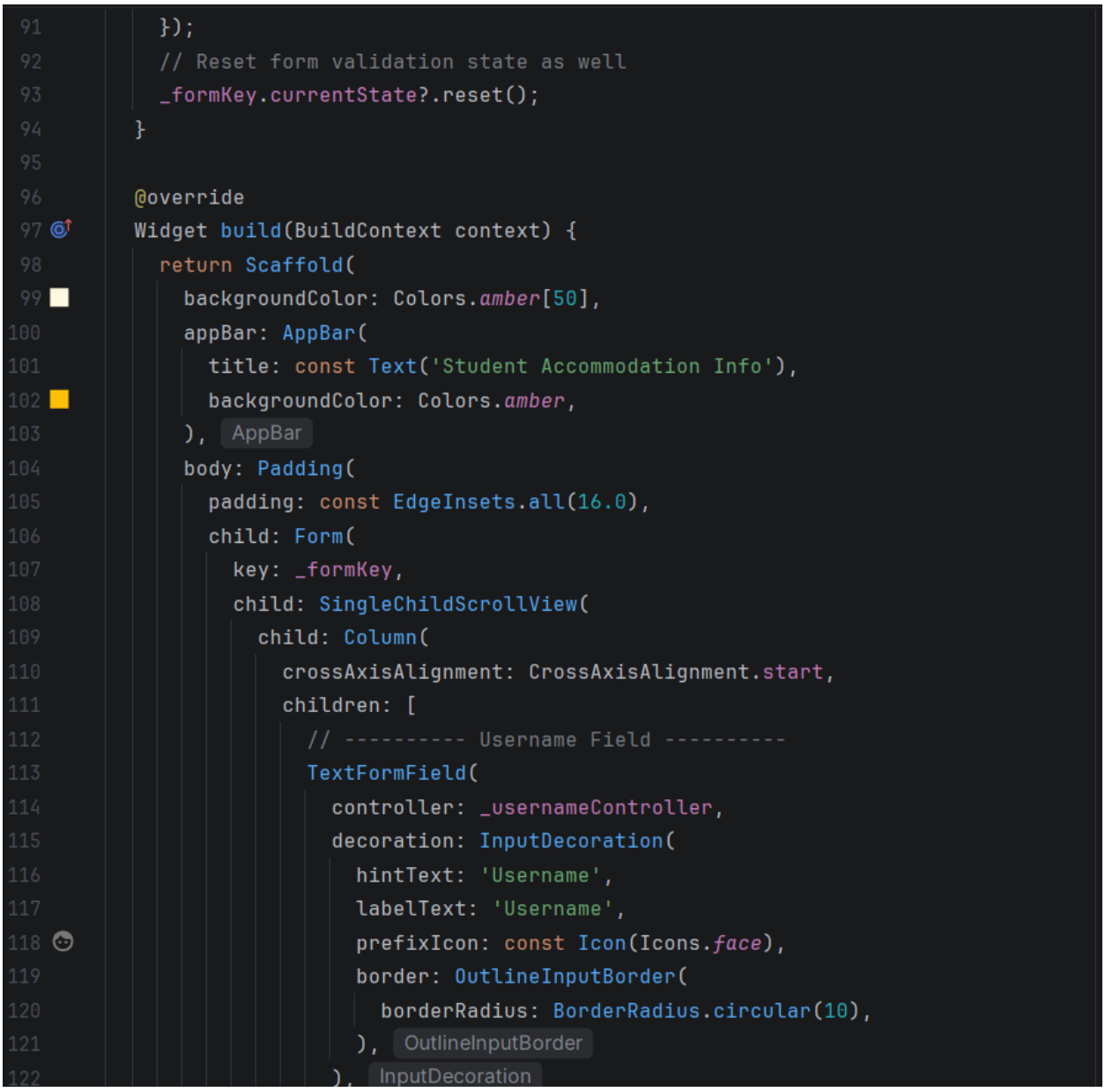
  // Checkbox state - whether the student has a special accommodation request
  bool _hasSpecialRequest = false;

  // Message shown after submitting the form
  String _resultMessage = '';

  @override
  void dispose() {
    // Dispose controllers to avoid memory leaks
    _usernameController.dispose();
    _passwordController.dispose();
    _accommodationDetailsController.dispose();
    super.dispose();
  }

  // Called when the Submit button is pressed
  void _submitForm() {
    // Run all validators attached to the form fields
```

```
61      void _submitForm() {
62        // Run all validators attached to the form fields
63        if (_formKey.currentState!.validate()) {
64          setState(() {
65            String laptopText = _hasLaptop == 'Yes' ? 'do' : "don't";
66            String isText = _isStudent == 'Yes' ? '' : 'not ';
67
68            _resultMessage =
69            'Your username is ${_usernameController.text}. You $laptopText have Laptop\n'
70                'You are ${isText}a IS student at UCSC';
71
72            if (_hasSpecialRequest &&
73                _accommodationDetailsController.text.isNotEmpty) {
74              _resultMessage +=
75              '\nSpecial request: ${_accommodationDetailsController.text}';
76            }
77          });
78        }
79      }
80
81      // Called when the Clear button is pressed
82      void _clearForm() {
83        setState(() {
84          _usernameController.clear();
85          _passwordController.clear();
86          _accommodationDetailsController.clear();
87          _hasLaptop = 'Yes';
88          _isStudent = 'Yes';
89          _hasSpecialRequest = false;
90          _resultMessage = '';
91        });
92        // Reset form validation state as well
```



```
    });
    // Reset form validation state as well
    _formKey.currentState?.reset();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.amber[50],
      appBar: AppBar(
        title: const Text('Student Accommodation Info'),
        backgroundColor: Colors.amber,
      ), AppBar
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Form(
          key: _formKey,
          child: SingleChildScrollView(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                // ---------- Username Field ----------
                TextFormField(
                  controller: _usernameController,
                  decoration: InputDecoration(
                    hintText: 'Username',
                    labelText: 'Username',
                    prefixIcon: const Icon(Icons.face),
                    border: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(10),
                    ), OutlineInputBorder
                  ), InputDecoration
```

```
121                 ), OutlineInputBorder
122               ), InputDecoration
123               validator: (value) {
124                 if (value == null || value.trim().isEmpty) {
125                   return 'Username cannot be empty';
126                 }
127                 return null;
128               },
129             ), TextFormField
130             const SizedBox(height: 16),
131
132             // ---------- Password Field ----------
133             TextFormField(
134               controller: _passwordController,
135               obscureText: true, // hide the password text
136               decoration: InputDecoration(
137                 hintText: 'Password',
138                 labelText: 'Password',
139 ***             prefixIcon: const Icon(Icons.password),
140                 border: OutlineInputBorder(
141                   borderRadius: BorderRadius.circular(10),
142                 ), OutlineInputBorder
143               ), InputDecoration
144               validator: (value) {
145                 if (value == null || value.length < 8) {
146                   return 'Password must be at least 8 characters';
147                 }
148                 return null;
149               },
150             ), TextFormField
151             const SizedBox(height: 20),
152
```

```dart
const SizedBox(height: 20),

// ---------- Laptop Radio Buttons ----------
const Text(
  'Do you have a Laptop?',
  style: TextStyle(fontWeight: FontWeight.bold),
), Text
Row(
  children: [
    Expanded(
      child: RadioListTile<String>(
        title: const Text('Yes'),
        value: 'Yes',
        groupValue: _hasLaptop,
        onChanged: (value) {
          setState(() => _hasLaptop = value);
        },
      ), RadioListTile
    ), Expanded
    Expanded(
      child: RadioListTile<String>(
        title: const Text('No'),
        value: 'No',
        groupValue: _hasLaptop,
        onChanged: (value) {
          setState(() => _hasLaptop = value);
        },
      ), RadioListTile
    ), Expanded
  ],
), Row
const SizedBox(height: 10),
```

```
),  Row
const SizedBox(height: 10),

// ---------- IS Student Dropdown ----------
const Text(
  'Are you an IS student?',
  style: TextStyle(fontWeight: FontWeight.bold),
),  Text
DropdownButtonFormField<String>(
  value: _isStudent,
  decoration: InputDecoration(
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(10),
    ),  OutlineInputBorder
  ),  InputDecoration
  items: const [
    DropdownMenuItem(value: 'Yes', child: Text('Yes')),
    DropdownMenuItem(value: 'No', child: Text('No')),
  ],
  onChanged: (value) {
    setState(() => _isStudent = value);
  },
),  DropdownButtonFormField
const SizedBox(height: 20),

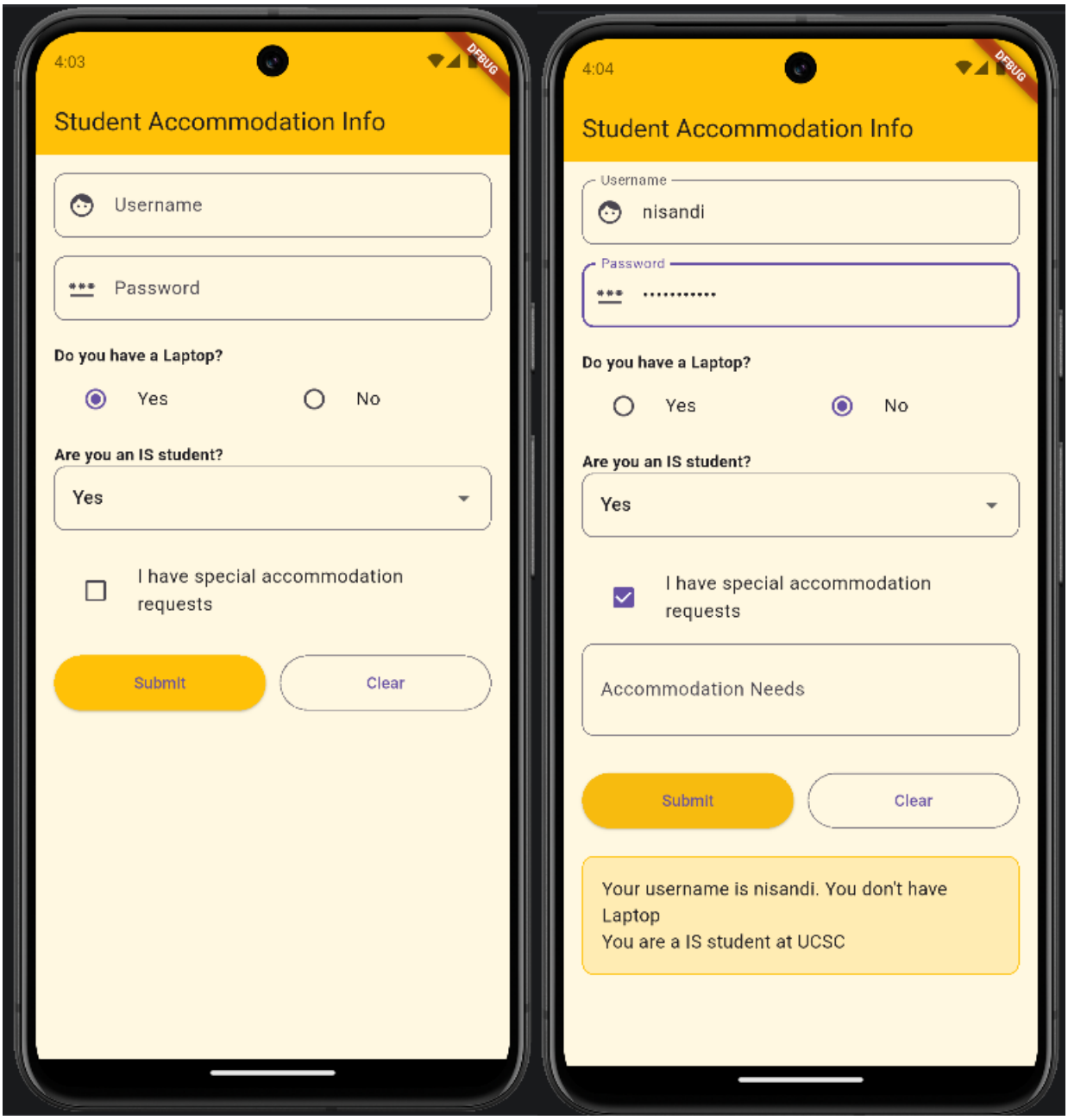
// ---------- Special Accommodation Request Checkbox ----------
CheckboxListTile(
  title: const Text('I have special accommodation requests'),
  value: _hasSpecialRequest,
  controlAffinity: ListTileControlAffinity.leading,
  onChanged: (value) {
    setState(() {
```

```
211                   onChanged: (value) {
212                     setState(() {
213                       _hasSpecialRequest = value ?? false;
214                     });
215                   },
216                 ), CheckboxListTile

217
218                 // Show this field only when the checkbox is checked
219                 if (_hasSpecialRequest)
220                   Padding(
221                     padding: const EdgeInsets.only(top: 8.0, bottom: 8.0),
222                     child: TextFormField(
223                       controller: _accommodationDetailsController,
224                       maxLines: 2,
225                       decoration: InputDecoration(
226                         hintText: 'Describe your accommodation needs',
227                         labelText: 'Accommodation Needs',
228                         border: OutlineInputBorder(
229                           borderRadius: BorderRadius.circular(10),
230                         ), OutlineInputBorder
231                       ), InputDecoration
232                     ), TextFormField
233                   ), Padding
234                 const SizedBox(height: 24),
235
236                 // ---------- Submit & Clear Buttons ----------
237                 Row(
238                   children: [
239                     Expanded(
240                       child: ElevatedButton(
241                         style: ElevatedButton.styleFrom(
242                           backgroundColor: Colors.amber,
```

```
244                     ),
245                     onPressed: _submitForm,
246                     child: const Text('Submit'),
247                   ), ElevatedButton
248                 ), Expanded
249                 const SizedBox(width: 12),
250                 Expanded(
251                   child: OutlinedButton(
252                     style: OutlinedButton.styleFrom(
253                       padding: const EdgeInsets.symmetric(vertical: 14),
254                     ),
255                     onPressed: _clearForm,
256                     child: const Text('Clear'),
257                   ), OutlinedButton
258                 ), Expanded
259               ],
260             ), Row
261             const SizedBox(height: 24),
262
263             // ---------- Result Message ----------
264             if (_resultMessage.isNotEmpty)
265               Container(
266                 width: double.infinity,
267                 padding: const EdgeInsets.all(16),
268                 decoration: BoxDecoration(
269                   color: Colors.amber[100],
270                   borderRadius: BorderRadius.circular(10),
271                   border: Border.all(color: Colors.amber),
272                 ), BoxDecoration
273                 child: Text(
274                   _resultMessage,
275                   style: const TextStyle(fontSize: 16),
```

```
259                   ],
260                 ), Row
261                 const SizedBox(height: 24),
262
263                 // ---------- Result Message ----------
264                 if (_resultMessage.isNotEmpty)
265                   Container(
266                     width: double.infinity,
267                     padding: const EdgeInsets.all(16),
268                     decoration: BoxDecoration(
269                       color: Colors.amber[100],
270                       borderRadius: BorderRadius.circular(10),
271                       border: Border.all(color: Colors.amber),
272                     ), BoxDecoration
273                     child: Text(
274                       _resultMessage,
275                       style: const TextStyle(fontSize: 16),
276                     ), Text
277                   ), Container
278                 ],
279               ), Column
280             ), SingleChildScrollView
281           ), Form
282         ), Padding
283       ); Scaffold
284     }
285   }
```


4:03
DEBUG
Student Accommodation Info
Username
Password
Do you have a Laptop?
Yes
No
Are you an IS student?
Yes
I have special accommodation requests
Submit
Clear
4:04
DEBUG
Student Accommodation Info
Username
nisandi
Password
Do you have a Laptop?
Yes
No
Are you an IS student?
Yes
I have special accommodation requests
Accommodation Needs
Submit
Clear
Your username is nisandi. You don't have Laptop
You are a IS student at UCSC

02)

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'To-Do List App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ), ThemeData
      home: const ToDoListScreen(),
    ); MaterialApp
  }
}

// Simple model class representing a single task
class Task {
  String title;
  bool isCompleted;

  Task({required this.title, this.isCompleted = false});
}

class ToDoListScreen extends StatefulWidget {
  const ToDoListScreen({super.key});

```

```dart
  const ToDoListScreen({super.key});

  @override
  State<ToDoListScreen> createState() => _ToDoListScreenState();
}

class _ToDoListScreenState extends State<ToDoListScreen> {
  // Controller for the text input field
  final TextEditingController _taskController = TextEditingController();

  // List that holds all the tasks
  final List<Task> _tasks = [];

  @override
  void dispose() {
    _taskController.dispose();
    super.dispose();
  }

  // Add a new task to the list
  void _addTask() {
    final text = _taskController.text.trim();
    if (text.isEmpty) return; // don't add empty tasks

    setState(() {
      _tasks.add(Task(title: text));
    });

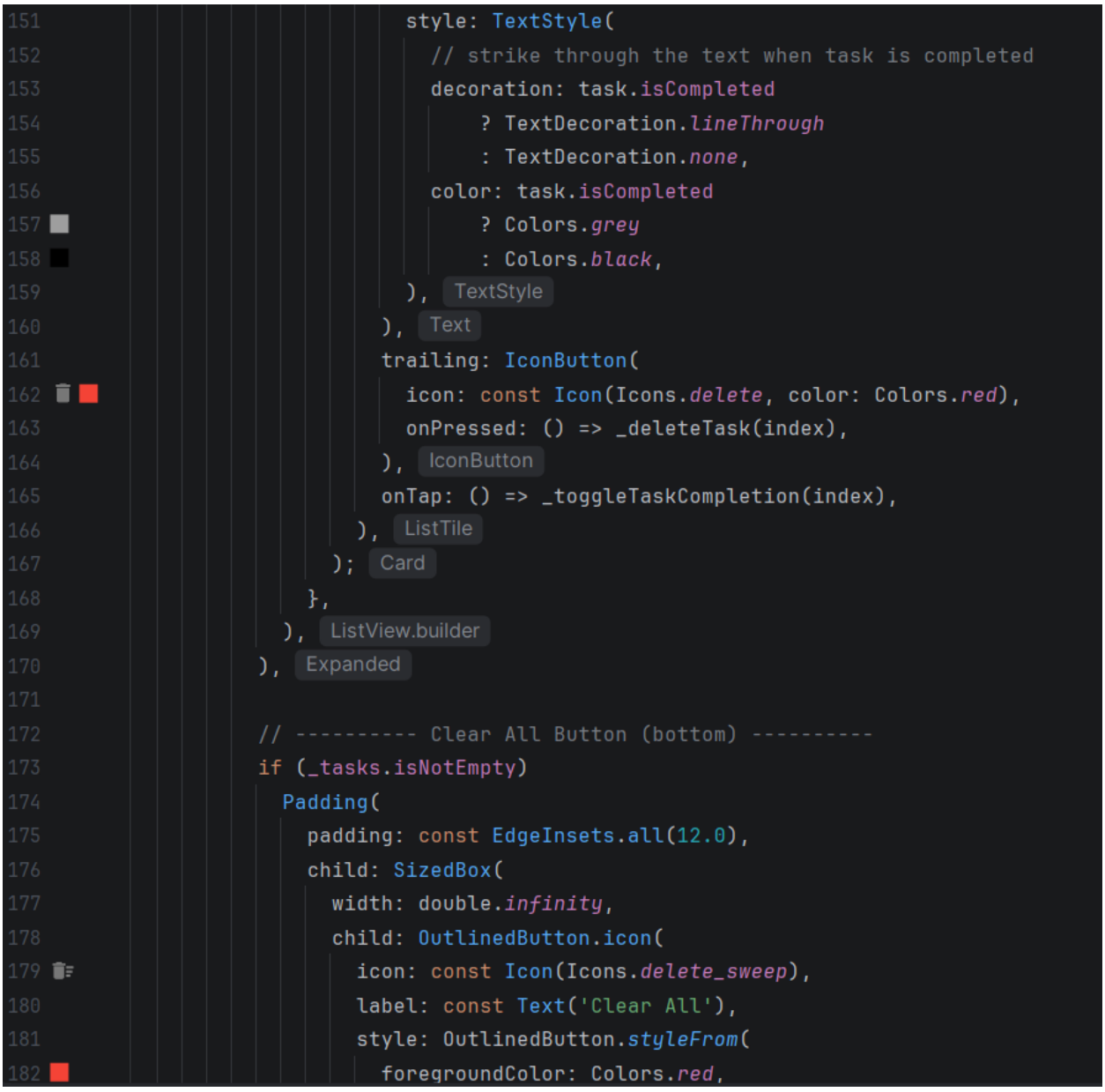
    _taskController.clear(); // clear the input field after adding
  }

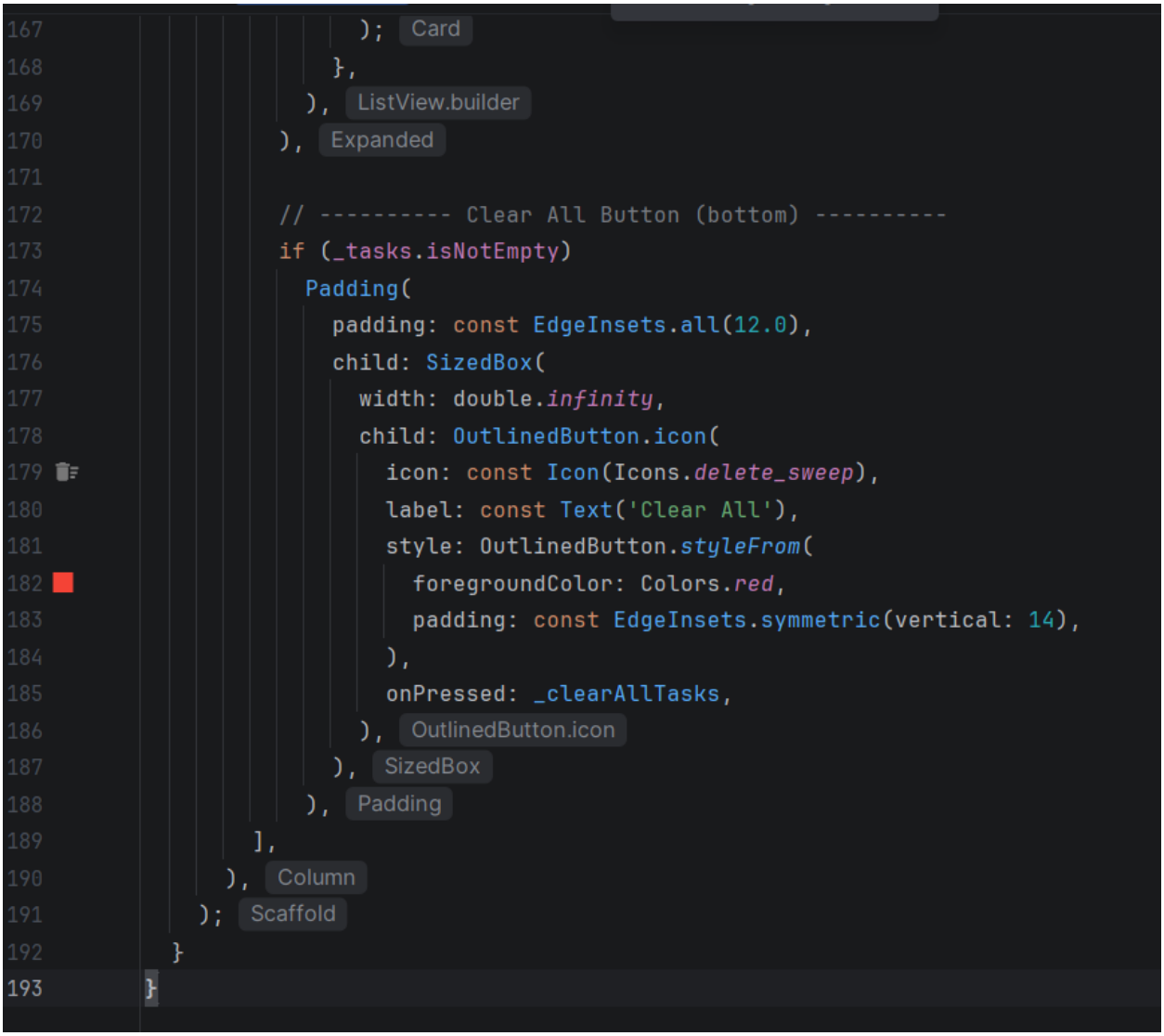
  // Toggle the completed state of a task
```

```dart
61
62    // Toggle the completed state of a task
63    void _toggleTaskCompletion(int index) {
64      setState(() {
65        _tasks[index].isCompleted = !_tasks[index].isCompleted;
66      });
67    }
68
69    // Delete a single task
70    void _deleteTask(int index) {
71      setState(() {
72        _tasks.removeAt(index);
73      });
74    }
75
76    // Delete all tasks
77    void _clearAllTasks() {
78      setState(() {
79        _tasks.clear();
80      });
81    }
82
83    @override
84    Widget build(BuildContext context) {
85      return Scaffold(
86        appBar: AppBar(
87          title: const Text('To-Do List'),
88          actions: [
89            // Clear All button in the app bar
90            IconButton(
91              icon: const Icon(Icons.delete_sweep),
92              tooltip: 'Clear All',
```

```
            icon: const Icon(Icons.delete_sweep),
            tooltip: 'Clear All',
            onPressed: _tasks.isEmpty ? null : _clearAllTasks,
          ), IconButton
        ],
      ), AppBar
      body: Column(
        children: [
          // ---------- Input Field + Add Button ----------
          Padding(
            padding: const EdgeInsets.all(12.0),
            child: Row(
              children: [
                Expanded(
                  child: TextField(
                    controller: _taskController,
                    decoration: InputDecoration(
                      hintText: 'Enter a new task',
                      border: OutlineInputBorder(
                        borderRadius: BorderRadius.circular(10),
                      ), OutlineInputBorder
                    ), InputDecoration
                    // allow adding by pressing enter/done on the keyboard
                    onSubmitted: (_) => _addTask(),
                  ), TextField
                ), Expanded
                const SizedBox(width: 8),
                ElevatedButton(
                  onPressed: _addTask,
                  child: const Text('Add'),
                ), ElevatedButton
              ],
```

```
121                     ), ElevatedButton
122                   ],
123                 ), Row
124               ), Padding
125
126               // ---------- Task List ----------
127               Expanded(
128                 child: _tasks.isEmpty
129                     ? const Center(
130                   child: Text(
131                     'No tasks yet. Add one above!',
132                     style: TextStyle(color: Colors.grey),
133                   ), Text
134                 ) Center
135                     : ListView.builder(
136                   itemCount: _tasks.length,
137                   itemBuilder: (context, index) {
138                     final task = _tasks[index];
139                     return Card(
140                       margin: const EdgeInsets.symmetric(
141                         horizontal: 12,
142                         vertical: 6,
143                       ), EdgeInsets.symmetric
144                       child: ListTile(
145                         leading: Checkbox(
146                           value: task.isCompleted,
147                           onChanged: (_) => _toggleTaskCompletion(index),
148                         ), Checkbox
149                         title: Text(
150                           task.title,
151                           style: TextStyle(
152                             // strike through the text when task is completed
```

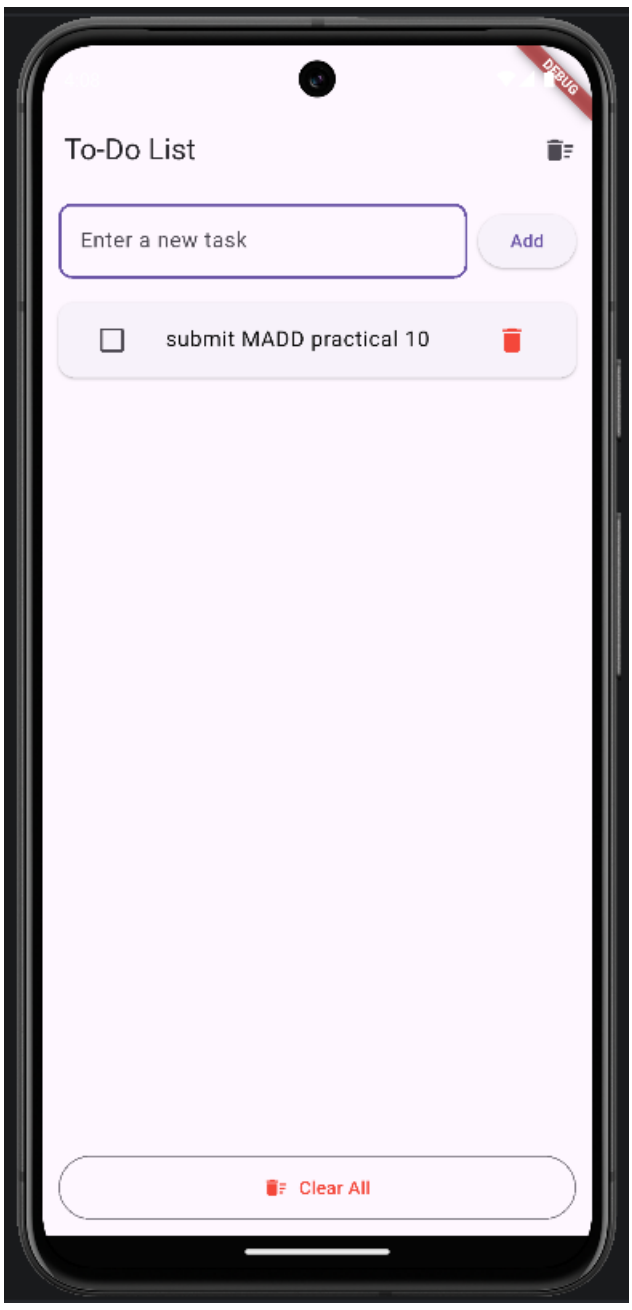

style: TextStyle(
// strike through the text when task is completed
decoration: task.isCompleted
? TextDecoration.lineThrough
: TextDecoration.none,
color: task.isCompleted
? Colors.grey
: Colors.black,
), TextStyle
), Text
trailing: IconButton(
icon: const Icon(Icons.delete, color: Colors.red),
onPressed: () => _deleteTask(index),
), IconButton
onTap: () => _toggleTaskCompletion(index),
), ListTile
); Card
},
), ListView.builder
), Expanded
// ---------- Clear All Button (bottom) ----------
if (_tasks.isNotEmpty)
Padding(
padding: const EdgeInsets.all(12.0),
child: SizedBox(
width: double.infinity,
child: OutlinedButton.icon(
icon: const Icon(Icons.delete_sweep),
label: const Text('Clear All'),
style: OutlinedButton.styleFrom(
foregroundColor: Colors.red,

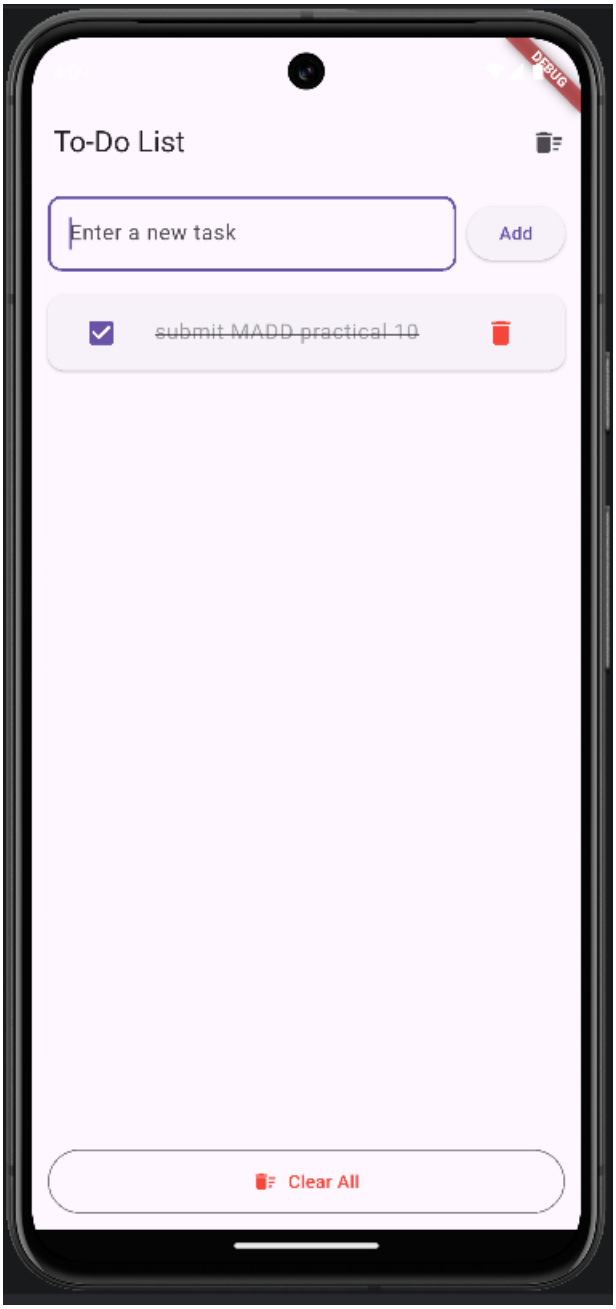


```
                    ); Card
                  },
                ), ListView.builder
              ), Expanded

              // ---------- Clear All Button (bottom) ----------
              if (_tasks.isNotEmpty)
                Padding(
                  padding: const EdgeInsets.all(12.0),
                  child: SizedBox(
                    width: double.infinity,
                    child: OutlinedButton.icon(
                      icon: const Icon(Icons.delete_sweep),
                      label: const Text('Clear All'),
                      style: OutlinedButton.styleFrom(
                        foregroundColor: Colors.red,
                        padding: const EdgeInsets.symmetric(vertical: 14),
                      ),
                      onPressed: _clearAllTasks,
                    ), OutlinedButton.icon
                  ), SizedBox
                ), Padding
            ],
          ), Column
        ); Scaffold
      }
}
```


DEBUG
To-Do List
Enter a new task
Add
No tasks yet. Add one above!


DEBUG
To-Do List
Enter a new task
Add
submit MADD practical 10
Clear All


DEBUG
To-Do List
Enter a new task
Add
submit MADD practical 10
Clear All